

내 손으로 조작하며 배우는 5일 강화학습

260204 3일차: Q-러닝과 Taxi-v3 정복

Q-Learning 알고리즘

- 1989년 크리스토퍼 왓킨스(Christopher Watkins)에 의해 처음 제안
- 주어진 환경에서 경험을 통해 최적 정책을 학습
 - 각 상태에서 어떤 행동을 취해야 최대 보상을 받는지 찾기
- Model-free: 환경의 모델(전이 확률, P)를 몰라도 학습이 가능

Q-Learning 알고리즘

- 세가지 구성요소
 - Q-값(Q-value)
 - 벨만 방정식(Bellman Equation)
 - Q-Table

Q-값 (Q-value)

- $Q(s,a)$
 - (s,a) : 현재 상태와 현재 행동의 쌍
 - 상태 s 에서 행동 a 를 취했을 때 기대되는 누적 보상

벨만 방정식 (Bellman Equation)

$$Q(s, a) = R + \gamma \max_{a'} Q(s', a')$$

- R : 즉각적인 보상
- γ (감마): 할인율 (0~1)
- (s', a') : 다음 상태와 다음 상태에서 가능한 모든 행동의 쌍
- $\max Q(s', a')$: 다음 상태의 최대 Q값

Q-Learning 알고리즘

- Q테이블의 Q값 업데이트를 위한 수식

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- $R + \gamma \max(Q(s', a'))$: 벨만 방정식
- $R + \gamma \max(Q(s', a')) - Q(s, a)$: 실제 받은 보상과 예측했던 보상의 차이
- α : 학습률

<https://youtu.be/lufECeWtN34>

학습률 (Learning Rate, α)

- 새로운 정보를 얼마나 빠르게 반영할 것인가
 - $\alpha = 0$: 학습하지 않음
 - $\alpha = 1$: 즉시 완전히 반영
 - 일반적 범위: 0.1 ~ 0.5

Q-테이블 (Q-Table)

- 모든 상태와 행동에 대한 Q값을 저장하는 테이블
- 행(Row): 상태 (State)
- 열(Column): 행동 (Action)
- 각 셀: $Q(s,a)$ 값
- 초기값: 0 또는 작은 랜덤 값

Q-테이블 (Q-Table)

상태	행동 1	행동 2	행동 3
S1	0.5	0.2	-0.1
S2	0.0	0.8	0.3
S3	-0.3	0.7	0.5

탐험 vs 활용

- 탐험 (Exploration)
 - 새로운 행동을 시도하여 더 나은 전략 발견
- 활용 (Exploitation)
 - 현재 알고 있는 최선의 행동 선택

ϵ -Greedy 전략 (Epsilon-Greedy)

- 탐험과 활용의 비율을 조절
- 탐험률 (Epsilon, ϵ)
 - 무작위 행동을 선택할 확률 (Exploration vs Exploitation)
 - 높은 ϵ : 더 많은 탐험
 - 낮은 ϵ : 학습된 정책 활용
- 학습이 진행됨에 따라 **epsilon** 값이 점진적으로 감소
 - 초기에는 탐험을 하고 후반에는 학습된 정보를 활용

Q-Learning 알고리즘

1. Q-테이블을 0 또는 작은 임의의 값으로 초기화
2. 현재 상태를 관찰
3. ϵ -Greedy 정책에 따라 행동 a 를 선택 (탐색 또는 활용)
4. 선택한 행동 a 를 수행하고 보상 r 과 새로운 상태 s' 를 관찰
5. 벨만 방정식을 사용해 Q-테이블의 Q-값을 업데이트
6. 3번으로 돌아가 반복

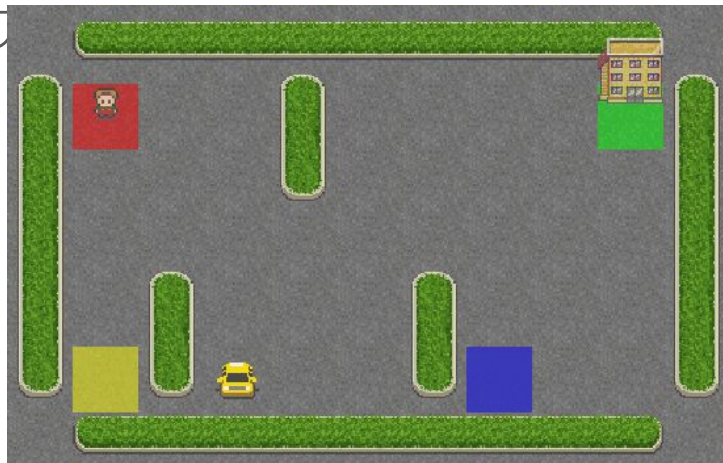
```
python day3/01_frozenlake_qlearning_pygame.py
```

```
python day3/01_frozenlake_qlearning_pygame.py --render_mode play --slippery
```

Taxi 환경

- 4개 정류장(R, G, Y, B) 중 한 곳에서 승객을 태워(Pickup) 지정된 다른 정류장에 내려주기
- 상태: 5x5 격자 공간 위치, 승객 탑승 여부, 승객의 목적지
- 행동: 상, 하, 좌, 우, 승객 태우기, 내리기
- 보상:
 - 성공 시 +20, 매 스텝 -1
 - 잘못된 승하차 시도 -10

`python day3/02_taxi_qlearning_pygame.py`



Q-Learning 알고리즘

- 주요 하이퍼파라미터
- 학습률 α : 0.1 ~ 0.8
- 할인율 γ : 0.9 ~ 0.99
- 탐험률 ϵ : 0.1 ~ 0.3
 - 탐험률 감소율: 0.9 ~ 0.99
- 에피소드 수: 1000 ~ 10000

```
streamlit run day3/03_qlearning_dashboard.py
```

Q-테이블 히트맵 시각화

- 학습된 Q값을 색상으로 표현
 - 밝은 색: 높은 Q값 (좋은 행동)
 - 어두운 색: 낮은 Q값 (나쁜 행동)
- 히트맵을 통한 학습 결과 확인
- 목표 상태로 향하는 경로 확인
- 학습이 잘 된 영역 vs 부족한 영역 파악
- 하이퍼파라미터 조정 방향 설정

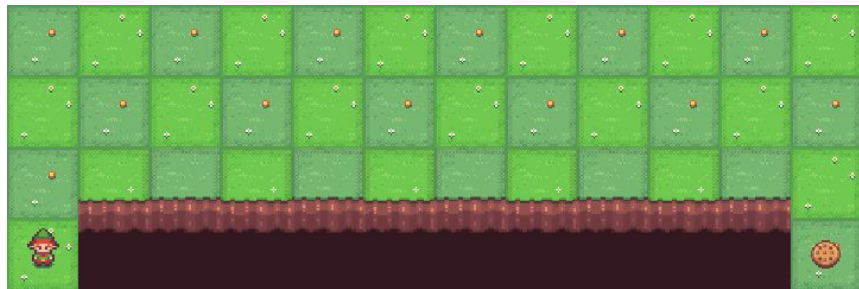
`day3/04_qtable_heatmap.ipynb`

Q-Learning 알고리즘의 한계

- 과도한 낙관성 (Overestimation) & 위험 감수 (Risk Taking)
 - 업데이트 식에 $\max$ 연산자가 있어 운 좋은 경험을 과대평가
 - 오프-폴리시(Off-Policy) 학습 특성상 "결과만 좋으면 과정은 상관없다"고 판단
 - 해결책:
 - SARSA(보수적인 선택지 중시)
 - Double DQN (선택과 평가 분리)

Cliff Walking 환경

- 위험 회피 및 최단 경로 찾기
- 상태: 4 x 12 격자 (시작점: 왼쪽 아래, 목적지: 오른쪽 아래)
- 행동: 상, 하, 좌, 우 4방향
- 보상:
 - 매 스텝 -1점 (최단 경로 유도)
 - 맨 아래 절벽: 즉시 종료, -100점



```
python day3/06_cliffwalking_qlearning_pygame.py
```

```
python day3/07_cliffwalking_sarsa_pygame.py
```

Q-Learning 알고리즘의 한계

- Q-테이블은 상태와 행동이 유한하고 이산적일 때만 가능
 - 상태가 복잡해지면 테이블 크기가 기하급수적으로 커져 생성 불가능.
(예: 아타리 게임)
 - 해결책: 4일차의 **DQN** (신경망 근사)
- 연속적인 환경 처리 불가 (Continuous Space)
 - '핸들 30.5도', '속도 100.3km/h' 같은 연속적인 값은 테이블에 기록할 수 없음.
 - 약한 해결책: 이산화(Discretization) -> 정밀도 하락.

python day3/08 mountaincar_continuous_compare.py

해결책: Policy Gradient (DDPG, PPO 등)

3일차 핵심 정리

- $Q(s,a)$: 상태-행동 쌍의 가치
- 벨만 방정식: Q 값 업데이트의 이론적 기반
- ϵ -greedy: 탐험과 활용의 균형
- Q-테이블: 학습 결과 저장 및 시각화

4일차 예고

- 인공신경망
- Deep Q-Network (DQN) 신경망을 이용한 Q값 근사
- Experience Replay (경험 재생) & Target Network

감사합니다